



Microcontroladores

Unidad 4: Periféricos

4.8: Interfaces de comunicación serial

Dr. Carlos Alberto Aguilar Avelar

Ing. en Computación

Facultad de Ingeniería

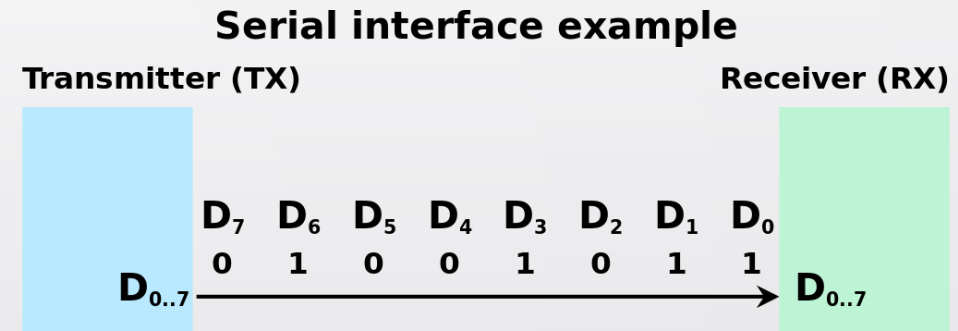
Universidad Autónoma de Baja California

Comunicación Serial

Transmisión de bits de manera consecutiva o secuencial, a través de un conductor simple denominado canal de comunicación, por el cual se transmite la información de acuerdo a un base de tiempo preestablecida.

Características:

1. Lenta
2. Compleja de implementar
3. Distancias largas (km)
4. Limitada por software
5. Económica y eficiente
6. Comunicaciones externas al MCU



Usada principalmente para el intercambio de información entre el microcontrolador y sus periféricos externos, aunque también puede ser utilizada para comunicarse con otros microcontroladores.

Comunicación Serial

Interface: Se denomina así al hardware o los componentes necesarios para llevar a cabo la comunicación, los cuales a su vez requieren de un canal de comunicación o conexión física entre los interlocutores para establecer la conexión.

Protocolo: Define las reglas para la transmisión de los datos a través de la interface de comunicación.

interface



protocolo

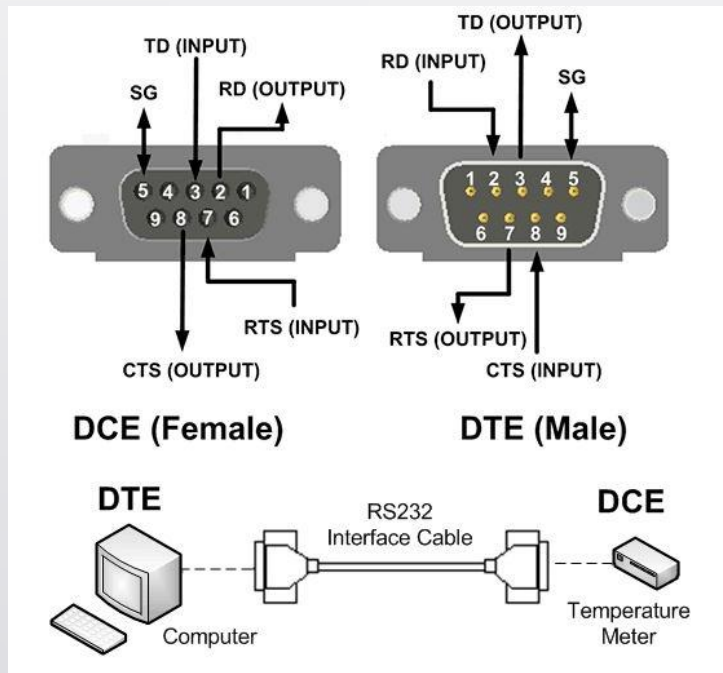


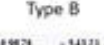
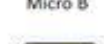
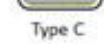
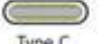
Por ejemplo, los protocolos de comunicación UART y SPI pueden ser implementados en la interface USART del ATmega328P.

Comunicación Serial

Estándar: Define tanto la interface y el protocolo que deben ser utilizados para establecer la comunicación entre dos interlocutores.

Por ejemplo, el estándar RS232 que era utilizado en las PCs antiguas y el estándar USB ampliamente utilizado hoy, ambos definen la interface y el protocolo utilizado para comunicar la PC con sus periféricos externos.



Interface Name	USB1.0	USB2.0	USB3.0 USB3.1 Gen1 USB3.2 Gen1	USB3.1 Gen2 USB3.2 Gen2	USB3.2 Gen2X2
Speed	1.5 Mbps	480 Mbps	5 Gbps	10 Gbps	20 Gbps
Logo					
Interface type	 Type A  Mini A  Micro A	 Type B  Mini B  Micro B	 Type A  Type B  Micro B  Type C	 Type A  Type C	 Type C

Comunicación Serial

Por el flujo de información entre los interlocutores, las interfaces seriales pueden ser:

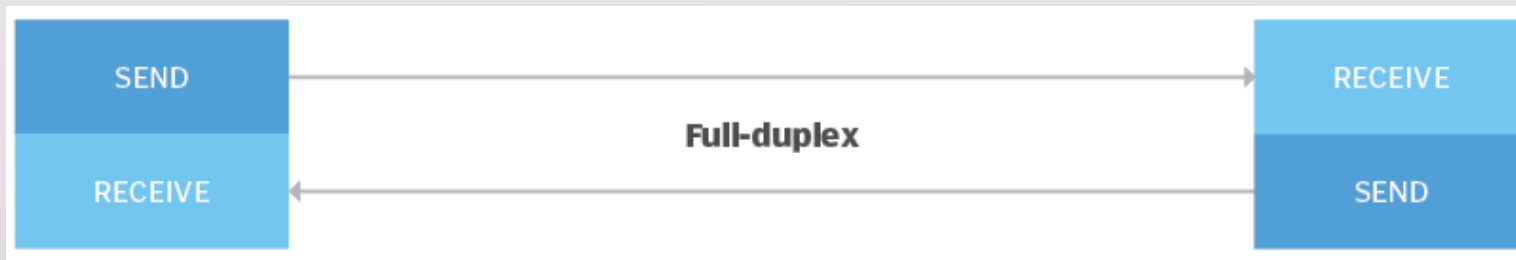
Simplex: Cuando la información solamente es transmitida en una sola dirección.



Half-duplex: Cuando la información se transmite en ambas direcciones, pero solamente es transmitida en una sola dirección a la vez.



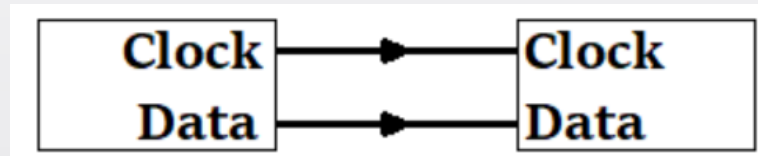
Full-duplex: Cuando la información se transmite en ambas direcciones simultáneamente.



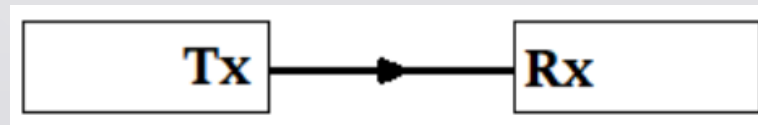
Comunicación Serial

Por el tipo de temporización o manejo de la señal de reloj, las interfaces seriales pueden ser:

Síncronas: Todos los interlocutores comparten la misma señal de reloj. El dispositivo encargado de generar la señal de reloj se le denomina Maestro, mientras quienes la reciben se conocen como Esclavos. La velocidad de transmisión puede ser más rápida y menos propensa a errores, pero requiere un canal exclusivo para la transmisión de la señal de reloj y es difícil transmitirla a largas distancias.



Asíncronas: Cada interlocutor se encarga de generar internamente sus señales de temporización, cuyos valores son preestablecidos. La velocidad de transmisión es más lenta y propensa a errores, por lo que se tienen que implementar métodos de detección y control de errores. Solo requiere los canales de datos, los cuales pueden ser transmitidos a distancias mayores.

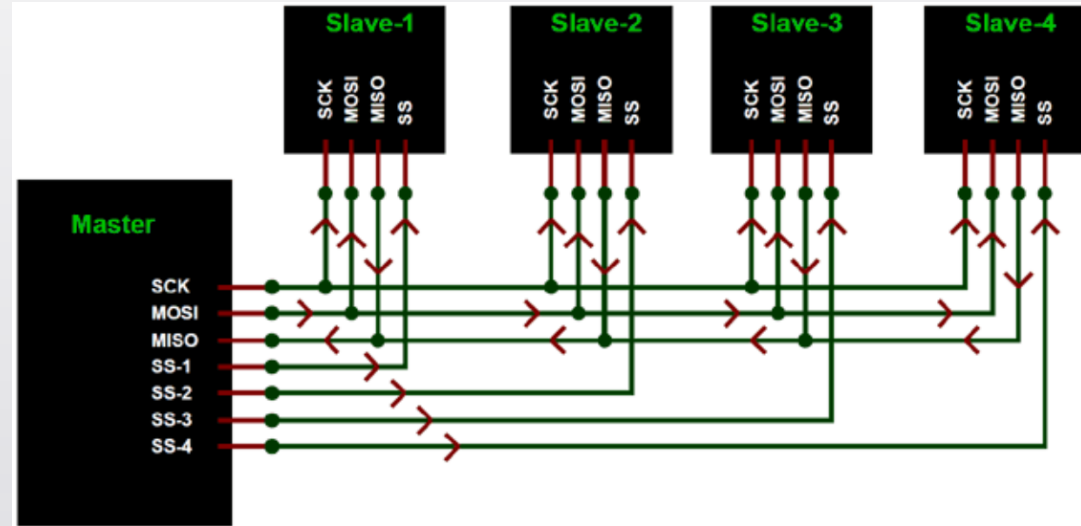


Serial Peripheral Interface (SPI)

Es un estándar de comunicaciones seriales que permite tener más de dos dispositivos interlocutores conectados a un mismo bus. Consta de una señal de reloj, un canal de transmisión, un canal de recepción y de una señal de selección por cada dispositivo con quien desea comunicarse. El maestro se encarga de generar la señal de reloj, establecer la señal de selección y de iniciar la comunicación.

Características:

- No usa bits de control o detección de errores.
- Los esclavos están limitados por la cantidad de señales de selección que el maestro pueda generar.
- La velocidad de transmisión puede llegar hasta los 10Mbps.
- Requiere al menos 4 líneas de transmisión.
- Es difícil establecer redes de transmisión.

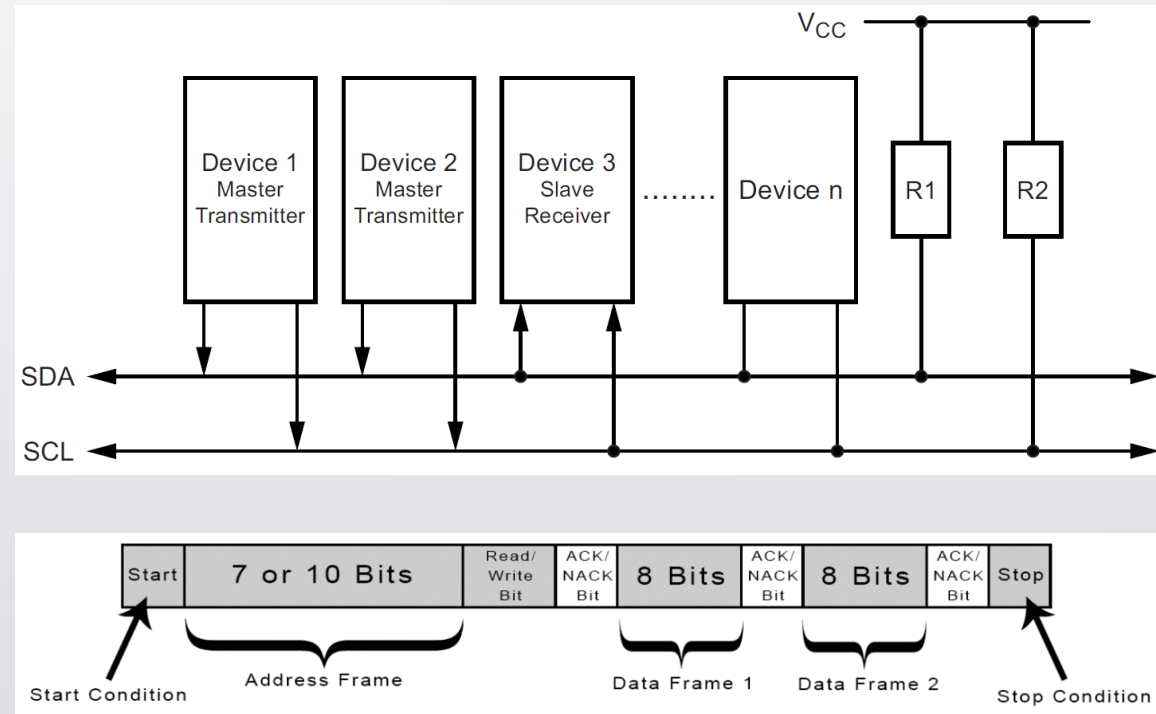


Inter Integrated Circuit (I2C)

Es una estándar de comunicaciones específicamente diseñado para comunicar circuitos integrados. Utiliza solo dos canales de transmisión, la línea de datos (SDA) y la línea de reloj (SCL), para comunicar uno o varios maestros con hasta 1024 dispositivos esclavos.

Características:

- Usa bits de control, detección de errores y de direccionamiento.
- Puede haber más de un maestro en el bus.
- Los esclavos están limitados por la cantidad de direcciones (10 bits) que permite el protocolo.
- La velocidad de transmisión va de los 400Kbps a los 5Mbps.
- Requiere solo 2 líneas de transmisión.
- Es fácil establecer redes de transmisión.



Otros estándares seriales conocidos



Universal Serial Bus (USB): Estándar que permite hasta 127 periféricos por cada dispositivo host. Fue diseñado para tener una interfaz simple y funcionamiento "plug&play". Usa un bus diferencial, lo que reduce la interferencia y permite altas velocidades de transmisión ($v_1=1.5\text{Mbps}$, $v_2=480\text{Mbps}$, $v_3=5\text{Gbps}$) a distancias cortas (5m).

Controller Area Network (CAN): Estándar diseñado por la industria automotriz para comunicar ECUs (Engine Control Units) y sensores. La interfaz es simple y el protocolo muy robusto, ya que permite detección de errores y envío de mensajes con IDs de prioridad. Todos los nodos de la red reciben los mensajes y deciden si es relevante. Puede alcanzar velocidades de 1Mbps y comunicar cientos de nodos a distancias de hasta 40 metros.

Recommended Standard 232 (RS232): Permite comunicación entre dos dispositivos a distancias de hasta 15 metros a 9600 baudios. Las líneas de transmisión y recepción envían de señales entre -3 y -15 V para 1 lógico y +3 y +15 V para 0 lógico.

RS422: Permite la comunicación entre un transmisor y hasta 10 receptores por medio de dos líneas diferenciales (4 cables). Puede llegar hasta los 10 Mbps a 10 metros. Los voltajes de las líneas de datos van de los -6V a los +6V, si su diferencia es mayor a +0.2 V se marca un cero lógico y 1 lógico para a una diferencia menor de -0.2V.

RS485: Interfaz que permite una red con 32 dispositivos transmisores/receptores con comunicación half-duplex y puede alcanzar una velocidad de 10 Mbps a 10 metros. El voltaje de la línea diferencial de datos va de los -7V a los +12V.

SPI en el ATmega328P

El Atmega328P cuenta con 1 interfaz SPI dedicada y el módulo USART puede funcionar como interfaz SPI adicional en modo Maestro.

SPI (Hardware)

USPIM (Hardware)

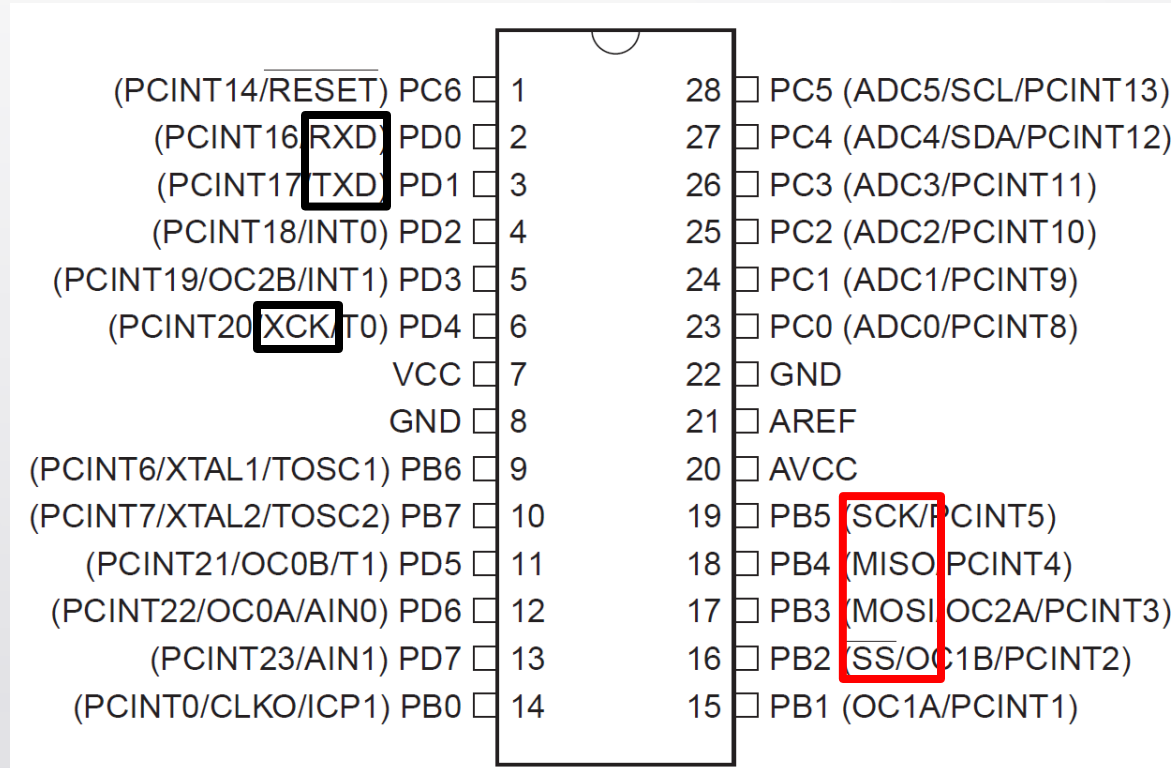
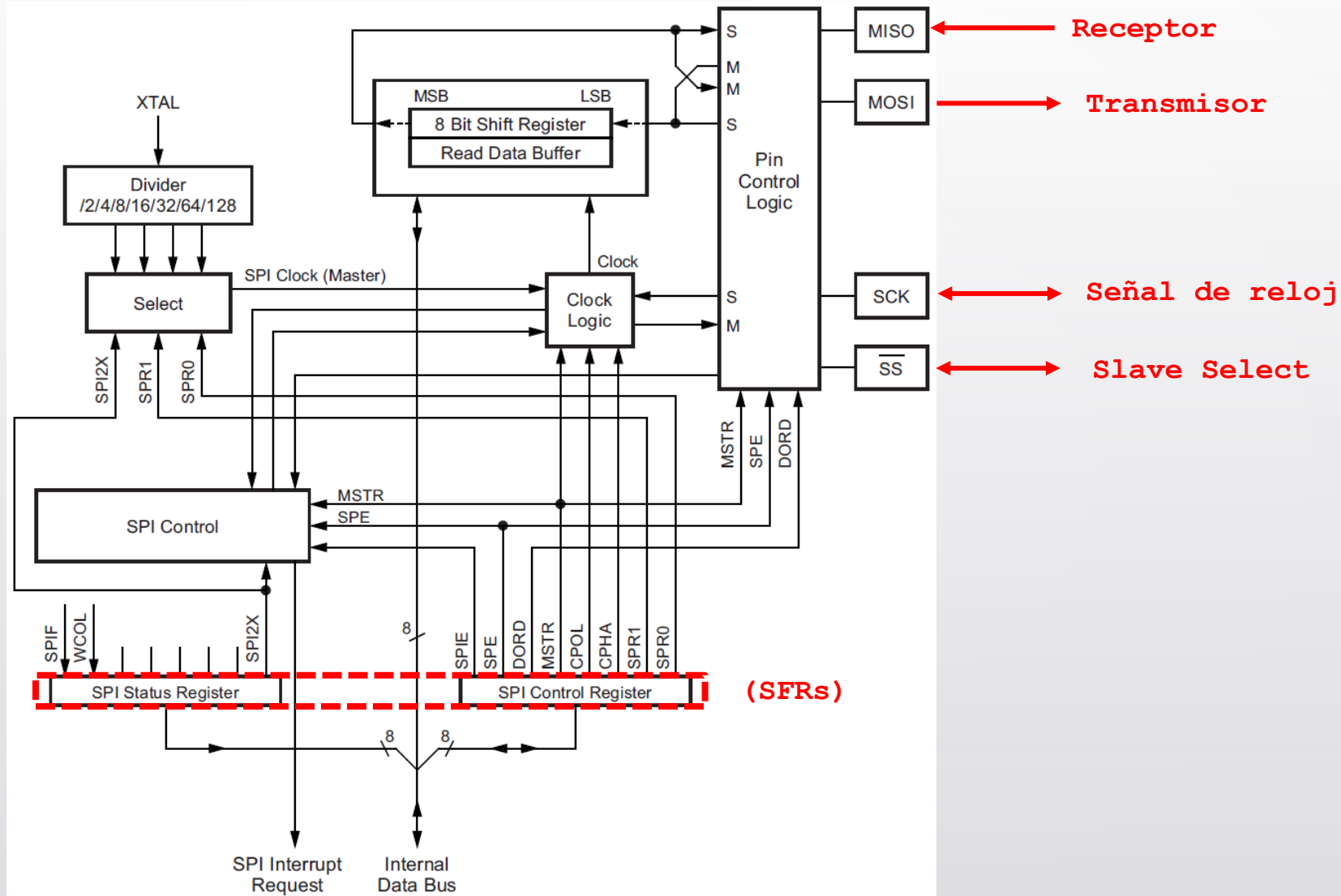


Diagrama del SPI



Registros del SPI

SPCR – SPI Control Register

Bit	7	6	5	4	3	2	1	0
0x2C (0x4C)	SPIE	SPE	DORD	MSTR	CPOL	CPHA	SPR1	SPR0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

Bit 7 – SPIE: SPI Interrupt Enable

Bit 6 – SPE: SPI Enable

Bit 5 – DORD: Data Order

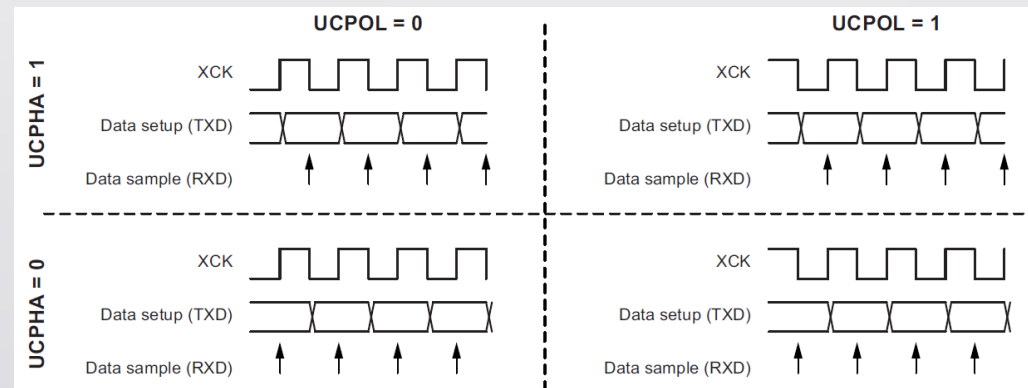
Bit 4 – MSTR: Master/Slave Select

Bit 3 – CPOL: Clock Polarity

Bit 2 – CPHA: Clock Phase

Bits 1, 0 – SPR1, SPR0: SPI Clock Rate Select 1 and 0

SPI Mode	Conditions
0	CPOL=0, CPHA=0
1	CPOL=0, CPHA=1
2	CPOL=1, CPHA=0
3	CPOL=1, CPHA=1



SPI2X	SPR1	SPR0	SCK Frequency
0	0	0	$f_{osc}/4$
0	0	1	$f_{osc}/16$
0	1	0	$f_{osc}/64$
0	1	1	$f_{osc}/128$
1	0	0	$f_{osc}/2$
1	0	1	$f_{osc}/8$
1	1	0	$f_{osc}/32$
1	1	1	$f_{osc}/64$

Registros del SPI

SPSR – SPI Status Register

Bit	7	6	5	4	3	2	1	0
0x2D (0x4D)	SPIF	WCOL	–	–	–	–	–	SPI2X
Read/Write	R	R	R	R	R	R	R	R/W
Initial Value	0	0	0	0	0	0	0	0

Bit 7 – SPIF: SPI Interrupt Flag

Bit 6 – WCOL: Write Collision Flag

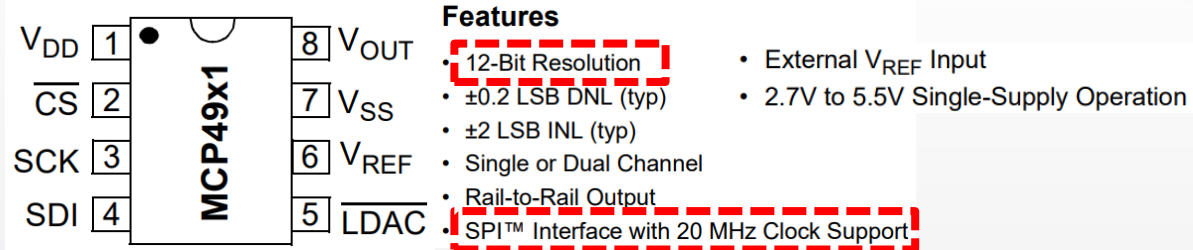
Bit 0 – SPI2X: Double SPI Speed Bit

SPDR – SPI Data Register

Bit	7	6	5	4	3	2	1	0
0x2E (0x4E)	MSB							LSB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	X	X	X	X	X	X	X	X

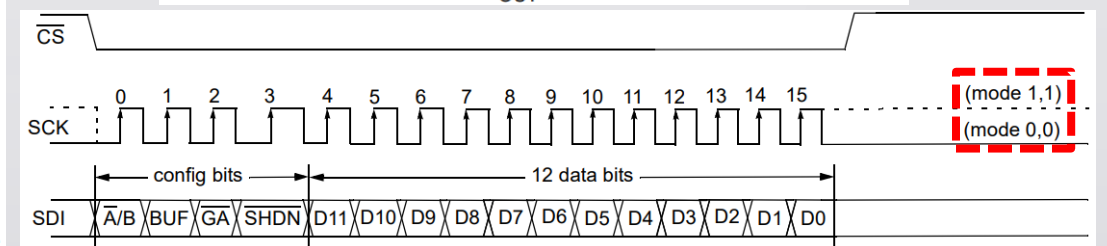
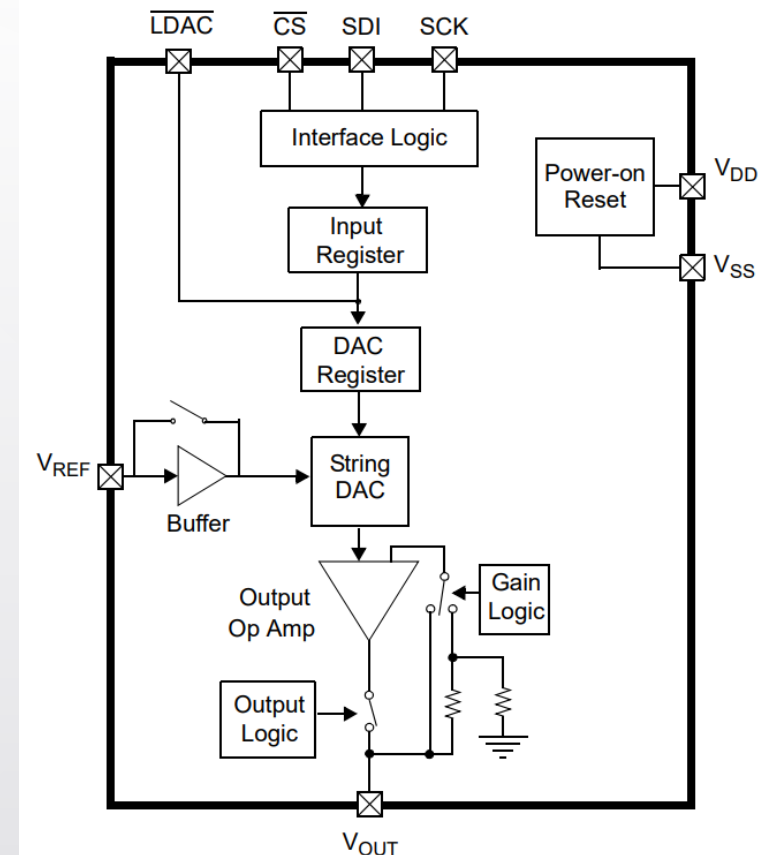
Bit 0:7 – SPI Data Register

12-bit DAC SPI (MCP4921)

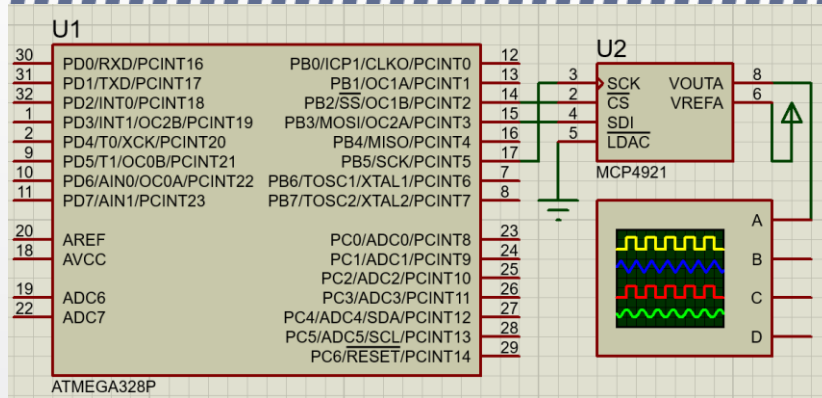


Upper Half:							
W-x	W-x	W-x	W-0	W-x	W-x	W-x	W-x
$\overline{A/B}$	BUF	$\overline{GA}$	$\overline{SHDN}$	D11	D10	D9	D8
bit 15				bit 8			
Lower Half:							
W-x	W-x	W-x	W-x	W-x	W-x	W-x	W-x
D7	D6	D5	D4	D3	D2	D1	D0
bit 7				bit 0			

- bit 15 **$\overline{A/B}$:** DAC_A or DAC_B Select bit
 1 = Write to DAC_B
 0 = Write to DAC_A
- bit 14 **BUF:** V_{REF} Input Buffer Control bit
 1 = Buffered
 0 = Unbuffered
- bit 13 **$\overline{GA}$:** Output Gain Select bit
 1 = 1x ($V_{OUT} = V_{REF} * D/4096$)
 0 = 2x ($V_{OUT} = 2 * V_{REF} * D/4096$)
- bit 12 **SHDN:** Output Power Down Control bit
 1 = Output Power Down Control bit
 0 = Output buffer disabled, Output is high impedance
- bit 11-0 **D11:D0:** DAC Data bits
 12 bit number "D" which sets the output value. Contains a value between 0 and 4095.



Ejemplo #1 - Generador de señal (MCP4921)



```
#include <avr/io.h>
#include "SPI.h"

void DAC_Output(uint16_t value)
{
    // 0:DACA; 0:BUFF; 1:GA(1x); 1=EN
    uint8_t data_high = 0b00110000;
    data_high |= 0x0f & (value>>8);
    uint8_t data_low = value;
    PORTB &= ~(1<<PINB2); // SS=LOW
    SPI_MasterTransmit(data_high);
    SPI_MasterTransmit(data_low);
    PORTB |= (1<<PINB2); // SS=HIGH
}

int main(void)
{
    // Initialize SPI Module
    SPI_MasterInit();
    // Counter for Sawtooth Wave
    uint16_t counter = 0;

    while (1)
    {
        DAC_Output(counter);
        counter++;
        if(counter>4095)counter=0;
    }
}
```

```
#ifndef SPI_H_
#define SPI_H_

void SPI_MasterInit();
void SPI_MasterTransmit(char);

void SPI_MasterInit(void)
{
    /* Set SS, MOSI and SCK as Output */
    DDRB |= (1<<PINB2)|(1<<PINB3)|(1<<PINB5);
    DDRB &= ~(1<<PINB4); // MISO as Input
    /* Set SPI Mode as 0 */
    SPCR |= (0<<CPOL)|(0<<CPHA);
    /* Set Data Order as MSF */
    SPCR |= (0<<DORD);
    /* Set clock rate fck/2 */
    SPCR |= (0<<SPR1)|(0<<SPR0);
    /* Enable SPI as Master */
    SPCR |= (1<<SPE)|(1<<MSTR);
    /* Set clock rate fck/2 */
    SPSR |= (1<<SPI2X);
    /* Set SPI Interrupt Enable */
    SPCR |= (0<<SPIE);
}

void SPI_MasterTransmit(char cData)
{
    /* Start transmission */
    SPDR = cData;
    /* Wait for transmission complete */
    while(!(SPSR & (1<<SPIF)));
}

#endif /* SPI_H_ */
```

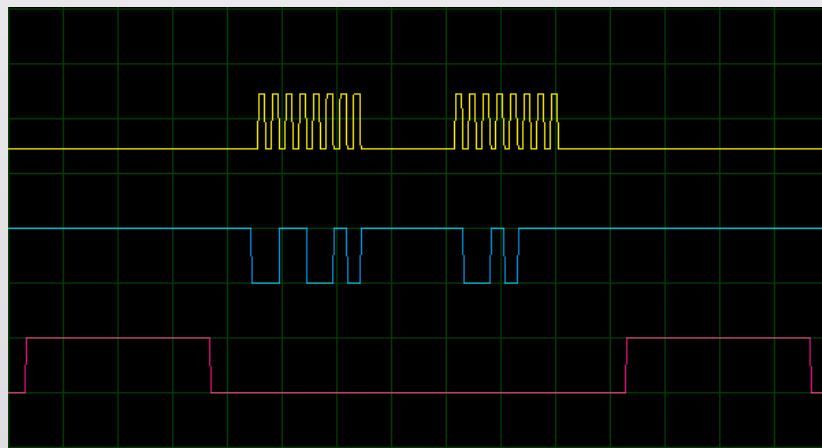
T = 22.5 ms
F = 44.44Hz



SCK

DATA

SS



Interrupciones del SPI

Vector de Interrupciones SPI

18	0x0022	SPI, STC	SPI serial transfer complete
19	0x0024	USART, RX	USART Rx complete
20	0x0026	USART, UDRE	USART, data register empty
21	0x0028	USART, TX	USART, Tx complete

SPCR – SPI Control Register

Bit	7	6	5	4	3	2	1	0
0x2C (0x4C)	SPIE	SPE	DORD	MSTR	CPOL	CPHA	SPR1	SPR0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

Bit 7 – SPIE: SPI Interrupt Enable (Transfer complete)

Actividad 4.8- Generador de funciones (SPI-DAC)

Utilizando la interface de comunicaciones seriales SPI del microcontrolador ATmega328P, diseñe un generador de funciones. Realice la simulación en Proteus e impleméntelo manera práctica utilizando el DAC MCP4921. La implementación debe contar con las siguientes características:

Especificaciones:

- Permite generar señales cuadradas, triangulares, diente de sierra y senoidales.
- El tipo de señal generada se selecciona con interruptores deslizables.
- Las señales generadas tienen una frecuencia fija de 100 Hz
- Las señales generadas tienen una amplitud fija de 5 Vpp.

**Bonus
Práctica #9**

